LAPORAN TUGAS KECIL 2

IF2211 STRATEGI ALGORITMA

VOXELIZATION OBJEK 3D MENGGUNAKAN OCTREE

Disusun oleh :

Muhamad Hasbullah Faris	18223014
Matthew Sebastian Kurniawan	18223096

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

TAHUN AJARAN 2025/2026

DAFTAR ISI

DAFTAR ISI 2
PERMASALAHAN 3
PENDEKATAN SOLUSI 3
IMPLEMENTASI SOLUSI 4
1. Struktur Data 4
2. Fungsi Parsing File OBJ 5
3. Fungsi Pembentukan AABB 8
4. Fungsi Pengecekan Interseksi (SAT) 9
5. Fungsi Pembangunan Octree 12
6. Fungsi Pengambilan Voxel 14
7. Fungsi Penulisan Output 15
8. Alur Utama Program 17
ANALISIS KOMPLEKSITAS 19
PENGUJIAN TEST CASE 20
Pengujian 1 (cow.obj) 20
Pengujian 2 (line.obj) 21
Pengujian 3 (pumpkin.obj) 22
Pengujian 4 (teapot.obj) 22
Pengujian 5 (teddybear.obj) 23
BONUS 23
1. Struktur Data Camera dan VoxelViewer 24
2. Operasi Vektor 25
3. Fungsi Posisi dan Basis Kamera 26
4. Fungsi Proyeksi 3D ke 2D 27
5. Konstruksi Geometri Box 28
6. Fungsi Update (Input Handling) 29
7. Fungsi Draw (Rendering) 30
8. Fungsi Layout 32
9. Fungsi Entry Point Viewer 32
PRANALA REPOSITORY 33
PERNYATAAN KEJUJURAN 33
LAMPIRAN 34
REFERENSI 34

PERMASALAHAN

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah struktur data hierarkis yang membagi ruang atau data menjadi bagian yang lebih kecil. Melalui Octree, sebuah kubus dalam ruang 3 dimensi dapat dibagi menjadi 8 anakan kubus berbeda, yang masing-masing anakannya dapat dibagi lagi menjadi 8 kubus yang lebih kecil.

PENDEKATAN SOLUSI

Dalam mengimplementasikan program ***Voxelization*** ini, akan digunakan pendekatan ***Divide and Conquer***, yaitu pendekatan yang akan membagi persoalan menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan semula namun berukuran lebih kecil, lalu menyelesaikan masing-masing upa-persoalan satu persatu. Setelahnya, solusi dari upa-persoalan tersebut akan digabungkan untuk memberikan solusi bagi persoalan semula.

Pada kasus ini, algoritma *Divide and Conquer* digunakan dalam membangun struktur data **Octree**. Pada pendekatan ini, mesh 3D beserta seluruh *vertices* dan *faces* dari file objek akan di-*wrap* oleh sebuah ruang **AABB (*Axis-Aligned Bounding Box*)**, kemudian proses *voxelization* dimulai dengan mengevaluasi *root node* untuk memeriksa apakah terdapat *faces* yang berpotensi *intersects* dengan AABB *bounds* melalui teorema **SAT *(Separating Axis Theorem)***. Jika *node* memiliki *triangles* yang *intersect* dan belum mencapai tingkat *depth* maksimal yang ditentukan, *node* tersebut akan dibagi menjadi 8 *sub-node* (*octants*) secara rekursif berdasarkan titik tengah bounds pada sumbu X, Y, dan Z. Dengan cara ini, *empty space* dapat diabaikan (*pruned*), sedangkan node yang *intersects* akan dibagi lagi menjadi 8 *sub-node* yang lebih rinci hingga mencapai batas kedalaman maksimum, di mana *node leaf* ditandai sebagai voxel.

Pendekatan ini memastikan efisiensi komputasi dengan meminimalkan jumlah voxel yang dihasilkan, sehingga mengurangi ukuran output file OBJ secara signifikan dibandingkan *grid uniform voxelization*. Selain itu, pembagian rekursif secara adaptif memungkinkan representasi detail yang lebih baik pada bagian mesh yang kompleks tanpa membuang *resources* pada *empty space*.

IMPLEMENTASI SOLUSI

1. Struktur Data

```go
type Vertex struct{
    X, Y, Z float64
}

// Representing faces of mesh
type Triangle struct{
    A, B, C Vertex // Vertices
}
type Mesh struct {
    ListOfVertex []Vertex
    Faces []Triangle
}

// Representing the space that wraps each node
type AABB struct{
    Min, Max Vertex // AABB defined by computing 2 opposite vertex of
mesh
}

// Representing the Tree Node
type TreeNode struct{
    Bounds AABB
    Children [8] * TreeNode
    IsVoxel bool
}

type Stats struct {
    NodesAtDepth []int
    PrunedAtDepth []int
    MaxDepth int
}
```

Dalam program didefinisikan beberapa struktur data utama, yaitu Vertex, Triangle, Mesh, AABB, TreeNode, dan Stats. **Vertex** merepresentasikan titik 3

dimensi dengan komponen X, Y, Z. **Triangle** merepresentasikan sebuah face pada objek mesh yang terdiri dari 3 Vertex. **Mesh** merepresentasikan keseluruhan objek 3 dimensi dengan komponen berupa *list* dari Vertex dan Faces. **AABB (*Axis-Aligned Bounding Box*)** merepresentasikan ruang kubus yang membungkus TreeNode dan dibatasi oleh titik Min dan Max. **TreeNode** merepresentasikan node pada pohon Octree dengan Bounds sebagai batas ruang, Children sebagai anakan *node*, dan IsVoxel sebagai penanda apakah *node* merupakah sebuah Voxels. **Stats** menyimpan statistik Octree berupa jumlah node, node yang dipangkas, dan kedalaman maksimum.

2. Fungsi Parsing File OBJ

```go
// Parsing the object
func parse(path string) (*Mesh, error){
    f, err := os.Open(path)

    if err != nil {
        return nil, err
    }

    defer f.Close()

    var listOfVertex []Vertex
    var faces []Triangle

    scanner := bufio.NewScanner(f)
    lineCount := 0

    for scanner.Scan(){
        lineCount += 1

        line := strings.TrimSpace(scanner.Text())
        fields := strings.Fields(line)

        if len(fields) == 0 || strings.HasPrefix(fields[0], "#") {
            continue
        }
```

```
        switch fields[0]{
            case "v":
                if len(fields) != 4{ // Input validation to check if
each line contain 4 item
                    return nil, fmt.Errorf("Input tidak valid pada
vertex ke %v!", lineCount)
                }

                v1, e1 := strconv.ParseFloat(fields[1], 64)
                v2, e2 := strconv.ParseFloat(fields[2], 64)
                v3, e3 := strconv.ParseFloat(fields[3], 64)

                if e1 != nil || e2 != nil || e3 != nil{
                    return nil, fmt.Errorf("Titik koordinat tidak valid
pada vertex ke %v!", lineCount)
                }

                listOfVertex = append(listOfVertex, Vertex{v1, v2, v3})

            case "f":
                if len(fields) != 4{
                    return nil, fmt.Errorf("Input tidak valid pada face
ke %v!", lineCount)
                }

                vertex1, e1 := strconv.Atoi(fields[1])
                vertex2, e2 := strconv.Atoi(fields[2])
                vertex3, e3 := strconv.Atoi(fields[3])

                if e1 != nil || e2 != nil || e3 != nil{
                    return nil, fmt.Errorf("Vertex tidak valid pada
face ke %v!", lineCount)
                }

                faces = append(faces, Triangle{A: listOfVertex[vertex1
- 1], B: listOfVertex[vertex2 - 1], C: listOfVertex[vertex3 - 1]})
```

```go
            default:
                continue
        }
    }
    // Scanning error validation
    err = scanner.Err()
    if err != nil{
        return nil, err
    }

    // Vertices and faces validation
    if len(listOfVertex) == 0 {
        return nil, fmt.Errorf("Tidak ditemukan vertices pada %s",
path)
    }
    if len(faces) == 0 {
        return nil, fmt.Errorf("Tidak ditemukan faces %s", path)
    }

    return &Mesh{ListOfVertex: listOfVertex, Faces: faces}, nil
}
```

Fungsi parse bertugas membaca file .obj dan mengubahnya menjadi struct Mesh. Pertama, file dibuka menggunakan os.Open, kemudian dibaca baris per baris menggunakan bufio.NewScanner, lali setiap baris di-*trim* berdasarkan spasi.

Selanjutnya, baris berawalan "v" diidentifikasi sebagai data Vertex, dan tiga nilai koordinatnya akan di-*parse* ke float64 dan dimasukkan ke dalam listOfVertex. Baris berawalan "f" diidentifikasi sebagai data Faces, dan tiga indeksnya digunakan untuk mendapatkan nilai vertex dari listOfVertex dan membentuk sebuah Triangle yang lalu ditambahkan ke *list* Faces. Terakhir, fungsi akan mengembalikan pointer ke Mesh yang berisi seluruh hasil parsing.

3. Fungsi Pembentukan AABB

```
// Forming the AABB box
func makeBox(mesh *Mesh) AABB{
    inf := math.Inf(1)
    box := AABB{Min: Vertex{inf, inf, inf}, Max: Vertex{-inf, -inf,
-inf}}

    // Wrap the mesh by defining the max and min vertex in 3 dimention
    for _, v := range mesh.ListOfVertex {
        if v.X < box.Min.X {
            box.Min.X = v.X
        }
        if v.Y < box.Min.Y {
            box.Min.Y = v.Y
        }
        if v.Z < box.Min.Z {
            box.Min.Z = v.Z
        }
        if v.X > box.Max.X {
            box.Max.X = v.X
        }
        if v.Y > box.Max.Y {
            box.Max.Y = v.Y
        }
        if v.Z > box.Max.Z {
            box.Max.Z = v.Z
        }
    }

    // Uniforming the wrap
    xLength := box.Max.X - box.Min.X
    yLength := box.Max.Y - box.Min.Y
    zLength := box.Max.Z - box.Min.Z

    // Choosing the highest length so all parts of mesh are covered
    side := math.Max(xLength, math.Max(yLength, zLength))
```

```
    xCenter := (box.Min.X + box.Max.X) * 0.5
    yCenter := (box.Min.Y + box.Max.Y) * 0.5
    zCenter := (box.Min.Z + box.Max.Z) * 0.5

    // Forming the box from the center point
    return AABB{Min: Vertex{xCenter - (0.5 * side), yCenter - (0.5 *
side), zCenter - (0.5 * side)}, Max: Vertex{xCenter + (0.5 * side),
yCenter + (0.5 * side), zCenter + (0.5 * side)}}
 }
```

Fungsi makeBox bertugas membentuk kubus AABB yang membungkus seluruh mesh. Variabel box diinisialisasi dengan Min bernilai positif tak hingga dan Max bernilai negatif tak hingga, kemudian dilakukan iterasi pada setiap vertex untuk memperbarui nilai minimum dan maksimum pada masing-masing sumbu X, Y, dan Z untuk mendapatkan nilai ekstrim nya.

Setelah itu, panjang sisi pada ketiga sumbu dihitung dan sisi terpanjang dipilih sebagai nilai sisi kubus sehingga dapat dipastikan kubus "membungkus" seluruh bagian Mesh. Selanjutnya, dihitung titik tengah dari tiap sisi, lalu dibangun kubus dari titik tengah tersebut ke tiap sudut kubus AABB. Terakhir, fungsi mengembalikan kubus AABB.

4. Fungsi Pengecekan Interseksi (SAT)

```
// Checking if the box wraps intersect with mesh using SAT Theorem
func checkIntersections(triangle Triangle, box AABB) bool{
    // Defining the new coordinates for triangle
    xCenter := (box.Min.X + box.Max.X) * 0.5
    yCenter := (box.Min.Y + box.Max.Y) * 0.5
    zCenter := (box.Min.Z + box.Max.Z) * 0.5

    xHalf := (box.Max.X - box.Min.X) * 0.5
    yHalf := (box.Max.Y - box.Min.Y) * 0.5
    zHalf := (box.Max.Z - box.Min.Z) * 0.5

    // Shift triangle so center of triangle is at the center of box
    newVertex1 := Vertex{triangle.A.X - xCenter, triangle.A.Y - yCenter,
```

```
triangle.A.Z - zCenter}
    newVertex2 := Vertex{triangle.B.X - xCenter, triangle.B.Y - yCenter,
triangle.B.Z - zCenter}
    newVertex3 := Vertex{triangle.C.X - xCenter, triangle.C.Y - yCenter,
triangle.C.Z - zCenter}

    // Compute the direction vectors of the triangle
    edge1 := Vertex{newVertex2.X - newVertex1.X, newVertex2.Y -
newVertex1.Y, newVertex2.Z - newVertex1.Z} // New Vertex 1 -> 2
    edge2 := Vertex{newVertex3.X - newVertex2.X, newVertex3.Y -
newVertex2.Y, newVertex3.Z - newVertex2.Z}
    edge3 := Vertex{newVertex1.X - newVertex3.X, newVertex1.Y -
newVertex3.Y, newVertex1.Z - newVertex3.Z}

    // isSeparated checks if an axis has a gap between the triangle and
the box
    // Projects all triangle vertices onto the axis, computes the box
projection radius, and checks if the triangle shadow falls outside of the
box radius
    isSeparated := func(ax, ay, az float64) bool {
        p0 := ax * newVertex1.X + ay * newVertex1.Y + az * newVertex1.Z
        p1 := ax * newVertex2.X + ay * newVertex2.Y + az * newVertex2.Z
        p2 := ax * newVertex3.X + ay * newVertex3.Y + az * newVertex3.Z
        r := xHalf * math.Abs(ax) + yHalf * math.Abs(ay) + zHalf *
math.Abs(az)

        if math.Max(p0, math.Max(p1, p2)) < -r || math.Min(p0,
math.Min(p1, p2)) > r{
            return true
        } else {
            return false
        }
    }

    // Cross product of each triangle edge with each box edge direction
(x, y, z)
    for _, e := range [3]Vertex{edge1, edge2, edge3} {
        if isSeparated(0, e.Z, -e.Y) {
```

```
            return false
        }
        if isSeparated(-e.Z, 0, e.X){
            return false
        }
        if isSeparated(e.Y, -e.X, 0){
            return false
        }
    }

    // Test 3 box face normals
    if isSeparated(1, 0, 0){
        return false
    }
    if isSeparated(0, 1, 0){
        return false
    }
    if isSeparated(0, 0, 1){
        return false
    }

    // Test the triangle face normal
    triangleNormal := Vertex{edge1.Y * edge2.Z - edge1.Z * edge2.Y,
edge1.Z * edge2.X - edge1.X * edge2.Z, edge1.X * edge2.Y - edge1.Y *
edge2.X}
    if isSeparated(triangleNormal.X, triangleNormal.Y,
triangleNormal.Z){
        return false
    }
    return true
 }
```

Fungsi checkIntersections memeriksa apakah segitiga saling *intersects* dengan kubus AABB menggunakan algoritma Separating Axis Theorem (SAT). Pertama, pusat AABB dihitung dan segitiga digeser agar pusat box berada di origin, sehingga menyederhanakan proses pengujian.

Menurut Separating Axis Theorem, 2 poligon konveks tidak berpotongan jika terdapat setidaknya terdapat 1 sumbu yang memisahkan proyeksi keduanya (Moller & Hainer, 2008). Dalam kasus ini, 2 poligonnya adalah segitiga dan kubus, maka dilakukan pengujian untuk menguji apakah terdapat setidaknya 1 sumbu dimana proyeksi keduanya terpisah. Terdapat 3 pengujian, yaitu:

a. **Pengujian terhadap sumbu normal sisi kubus:** Fungsi mengecek sumbu (1, 0, 0), (0, 1, 0), dan (0, 0, 1) yang mewakili arah normal masing-masing sisi kubus. Jika pada salah satu sumbu ini proyeksi segitiga sepenuhnya berada di luar proyeksi kubus, maka kedua bentuk tidak berpotongan dan fungsi langsung mengembalikan false.
b. **Pengujian terhadap sumbu normal bidang segitiga:** Fungsi menghitung normal bidang segitiga dengan mengambil hasil cross product dari dua edge segitiga, lalu menguji apakah proyeksi segitiga dan kubus terpisah pada sumbu normal ini. Jika terpisah, segitiga dan kubus tidak berpotongan dan fungsi mengembalikan false.
c. **Pengujian terhadap sumbu hasil *cross product* antara *edge* segitiga dan *edge* kubus:** Untuk setiap edge segitiga, fungsi menguji sumbu yang dihasilkan dari cross product antara edge tersebut dengan masing-masing arah edge kubus (x, y, z) melalui kombinasi seperti (0, e.Z, -e.Y), (-e.Z, 0, e.X), dan (e.Y, -e.X, 0). Jika pada salah satu sumbu ini proyeksi keduanya terpisah, fungsi langsung mengembalikan false.

Apabila pengujian , maka segitiga dan AABB dinyatakan berinterseksi dan fungsi mengembalikan true.

5. Fungsi Pembangunan Octree

```
 func makeTree(bounds AABB, triangles []Triangle, depth, maxDepth int,
stats *Stats) *TreeNode{
    // Store triangle that intersects with AABB box
    intersects := triangles[:0:0]
    for _, tri := range triangles {
        if checkIntersections(tri, bounds) {
            intersects = append(intersects, tri)
        }
    }
```

```go
    // If the node is empty, delete
    if len(intersects) == 0 {
        stats.PrunedAtDepth[depth] += 1
        return nil
    }
    stats.NodesAtDepth[depth] += 1

    // If this is the max depth, return node as a voxel
    if depth == maxDepth {
        return &TreeNode{Bounds: bounds, IsVoxel: true}
    }

    xCenter := (bounds.Min.X + bounds.Max.X) * 0.5
    yCenter := (bounds.Min.Y + bounds.Max.Y) * 0.5
    zCenter := (bounds.Min.Z + bounds.Max.Z) * 0.5

    // Form the children from the min, center, and max point of the box
    octants := [8]AABB{
        {bounds.Min, Vertex{xCenter, yCenter, zCenter}},
        {Vertex{xCenter, bounds.Min.Y, bounds.Min.Z},
Vertex{bounds.Max.X, yCenter, zCenter}},
        {Vertex{bounds.Min.X, yCenter, bounds.Min.Z}, Vertex{xCenter,
bounds.Max.Y, zCenter}},
        {Vertex{xCenter, yCenter, bounds.Min.Z}, Vertex{bounds.Max.X,
bounds.Max.Y, zCenter}},
        {Vertex{bounds.Min.X, bounds.Min.Y, zCenter}, Vertex{xCenter,
yCenter, bounds.Max.Z}},
        {Vertex{xCenter, bounds.Min.Y, zCenter}, Vertex{bounds.Max.X,
yCenter, bounds.Max.Z}},
        {Vertex{bounds.Min.X, yCenter, zCenter}, Vertex{xCenter,
bounds.Max.Y, bounds.Max.Z}},
        {Vertex{xCenter, yCenter, zCenter}, bounds.Max}}

    node := &TreeNode{Bounds: bounds}
    for i, oct := range octants {
        node.Children[i] = makeTree(oct, intersects, depth + 1,
maxDepth, stats)
```

```
    }
    return node
}
```

Fungsi makeTree merupakan inti dari algoritma Octree yang membangun pohon secara rekursif. Pada setiap pemanggilan, fungsi terlebih dahulu menyaring segitiga yang berpotongan dengan *bounds* menggunakan checkIntersections dan menyimpannya ke *slice* intersects.

Apabila intersects kosong, node dinyatakan tidak mengandung bagian *mesh*, sehingga dipangkas dengan mengembalikan nil dan statistik pemangkasan dicatat. Jika tidak kosong, jumlah *node* pada *depth* tersebut juga dicatat pada statistik.

Apabila kedalaman sudah mencapai maxDepth, node dijadikan voxel dengan mengembalikan TreeNode yang memiliki IsVoxel = true. Jika belum mencapai kedalaman maksimum, bounds dibagi menjadi 8 octant berdasarkan titik tengahnya, lalu fungsi dipanggil secara rekursif untuk setiap octant dengan kedalaman depth + 1 dan hanya meneruskan segitiga yang relevan (intersects). Setiap child dapat bernilai nil apabila tidak mengandung segitiga.

6. Fungsi Pengambilan Voxel

```
func getVoxels(node *TreeNode) []AABB{
    if node == nil{
        return nil
    }

    if node.IsVoxel{
        return []AABB{node.Bounds}
    }

    var allVoxels []AABB
    for _, child := range node.Children{
        allVoxels = append(allVoxels, getVoxels(child)...)
    }
```

```
    return allVoxels
}
```

Fungsi getVoxels mengumpulkan seluruh voxel dari pohon Octree yang telah dibangun secara rekursif dengan melakukan *traversal depth first*. Apabila node bernilai nil (node yang telah dipangkas), fungsi mengembalikan nil.

Apabila node adalah voxel (IsVoxel == true), Bounds dari node tersebut langsung dikembalikan sebagai *slice* dengan satu elemen. Jika node bukan keduanya, fungsi dipanggil rekursif untuk setiap child node dan seluruh hasil voxel dari masing-masing child tersebut digabungkan ke dalam satu *slice* menggunakan append dengan operator variadic ... . Apabila suatu child bernilai nil, pemanggilan rekursif akan menghasilkan nil yang tidak menambah isi *slice* saat diappend.

7. Fungsi Penulisan Output

```
func writeOutput(voxels []AABB, path string) (int, int, error) {
    f, err := os.Create(path)
    if err != nil {
        return 0, 0, err
    }
    defer f.Close()

    w := bufio.NewWriter(f)

    totalVertices := 0
    totalFaces := 0

    // Write every poxels
    for _, box := range voxels{
        // Write the vertices
        fmt.Fprintf(w, "v %g %g %g\n", box.Min.X, box.Min.Y, box.Min.Z)
// 0 LBF
        fmt.Fprintf(w, "v %g %g %g\n", box.Max.X, box.Min.Y, box.Min.Z)
// 1 RBF
        fmt.Fprintf(w, "v %g %g %g\n", box.Min.X, box.Max.Y, box.Min.Z)
```

```go
// 2 LTF
        fmt.Fprintf(w, "v %g %g %g\n", box.Max.X, box.Max.Y, box.Min.Z)
// 3 RTF
        fmt.Fprintf(w, "v %g %g %g\n", box.Min.X, box.Min.Y, box.Max.Z)
// 4 LBB
        fmt.Fprintf(w, "v %g %g %g\n", box.Max.X, box.Min.Y, box.Max.Z)
// 5 RBB
        fmt.Fprintf(w, "v %g %g %g\n", box.Min.X, box.Max.Y, box.Max.Z)
// 6 LTB
        fmt.Fprintf(w, "v %g %g %g\n", box.Max.X, box.Max.Y, box.Max.Z)
// 7 RTB

        b := totalVertices + 1 // OBJ index starts with 1

        // Write the faces
        faces := [12][3]int{
            {b + 0, b + 4, b + 5},
            {b + 0, b + 5, b + 1},
            {b + 2, b + 3, b + 7},
            {b + 2, b + 7, b + 6},
            {b + 0, b + 1, b + 3},
            {b + 0, b + 3, b + 2},
            {b + 4, b + 6, b + 7},
            {b + 4, b + 7, b + 5},
            {b + 0, b + 2, b + 6},
            {b + 0, b + 6, b + 4},
            {b + 1, b + 5, b + 7},
            {b + 1, b + 7, b + 3}}

        for _, face := range faces {
            fmt.Fprintf(w, "f %d %d %d\n", face[0], face[1], face[2])
        }

        totalVertices += 8
        totalFaces += 12
    }

    return totalVertices, totalFaces, w.Flush()
```

```
}
```

Fungsi writeOutput menyimpan hasil voxelisasi ke dalam file .obj. File dibuat menggunakan os.Create dan penulisan dilakukan dengan bufio.NewWriter untuk efisiensi. Untuk setiap voxel, fungsi menuliskan 8 vertex yang merepresentasikan sudut-sudut kubus, mulai dari LBF (*Left-Bottom-Front*) hingga RTB (*Right-Top-Back*).

Selanjutnya, fungsi menuliskan 12 face (segitiga) yang merepresentasikan 6 sisi kubus, di mana setiap sisi terdiri dari 2 segitiga. Indeks vertex menggunakan variabel b karena format OBJ menggunakan indeks berbasis 1 yang bersifat global. Setelah seluruh data ditulis, buffer dikosongkan dengan w.Flush(), dan fungsi mengembalikan jumlah total vertex, jumlah face, serta error jika ada.

8. Alur Utama Program

```
func main(){
    // Input validation
    if len(os.Args) != 3 {
        fmt.Fprintln(os.Stderr, "Format input: tucil2 <input.obj>
<max_depth>")
        os.Exit(1)
    }

    inputPath := os.Args[1]
    maxDepth, err := strconv.Atoi(os.Args[2])
    if err != nil || maxDepth < 1 {
        fmt.Fprintln(os.Stderr, "Error: tingkat kedalaman harus bernilai
positif!")
        os.Exit(1)
    }

    start := time.Now()

    mesh, err := parse(inputPath)
    if err != nil {
        fmt.Fprintf(os.Stderr, "Error: %v\n", err)
```

```go
        os.Exit(1)
    }

    bounds := makeBox(mesh)

    stats := &Stats{
        NodesAtDepth: make([]int, maxDepth+1),
        PrunedAtDepth: make([]int, maxDepth+1),
        MaxDepth:maxDepth,
    }

    root := makeTree(bounds, mesh.Faces, 1, maxDepth, stats)
    voxels := getVoxels(root)

    // Save solutions .obj
    outputPath := filepath.Join("./test/solution",
fmt.Sprintf("%s_voxel.obj", strings.TrimSuffix(filepath.Base(inputPath),
filepath.Ext(inputPath))))
    vertexCount, faceCount, err := writeOutput(voxels, outputPath)
    if err != nil {
        fmt.Fprintf(os.Stderr, "Terdapat error saat menyimpan solusi:
%v\n", err)
        os.Exit(1)
    }

    // Print stats
    fmt.Printf("Voxels         : %d\n", len(voxels))
    fmt.Printf("Vertices       : %d\n", vertexCount)
    fmt.Printf("Faces          : %d\n", faceCount)

    fmt.Println("Nodes formed per depth:")
    for d := 1; d <= stats.MaxDepth; d++ {
        fmt.Printf("  %d : %d\n", d, stats.NodesAtDepth[d])
    }
    fmt.Println()

    fmt.Println("Nodes not traversed per depth:")
    for d := 1; d <= stats.MaxDepth; d++ {
```

```
        fmt.Printf("  %d : %d\n", d, stats.PrunedAtDepth[d])
    }
    fmt.Println()

    fmt.Printf("Max depth      : %d\n", stats.MaxDepth)
    fmt.Printf("Execution time : %v\n", time.Since(start))
    fmt.Printf("Output file    : %s\n", outputPath)
}
```

Fungsi main merupakan titik masuk program yang diawali dengan validasi input, yaitu memastikan program menerima tepat dua argumen, path file .obj dan nilai max_depth. Nilai kedalaman dikonversi menggunakan strconv.Atoi dan harus bernilai positif. Selanjutnya, waktu eksekusi mulai dicatat dengan time.Now(), file input diproses menggunakan parse, dan bounding box (AABB) dibentuk melalui makeBox.

Struct Stats kemudian diinisialisasi untuk mencatat statistik pada setiap kedalaman. Pohon Octree dibangun secara rekursif menggunakan makeTree, voxel dikumpulkan dengan getVoxels, dan hasilnya disimpan ke file .obj melalui writeOutput. Terakhir, program mencetak berbagai statistik seperti jumlah voxel, vertex, face, distribusi node per kedalaman, waktu eksekusi, serta lokasi file output.

ANALISIS KOMPLEKSITAS

Dalam algoritma ini, proses utama yang mempengaruhi kompleksitas adalah pembentukan pohon melalui pembagian bounding box secara rekursif. Analisis kompleksitas dilakukan dengan melihat bagaimana fungsi makeTree membagi ruang 3 dimensi berdasarkan hasil uji perpotongan segitiga dengan AABB (*Axis-Aligned Bounding Box*) hingga mencapai batas kedalaman maksimum yang ditentukan.

Misalkan kedalaman maksimum octree adalah d dan jumlah segitiga pada mesh masukan adalah T. Setiap kali algoritma membagi suatu node, node tersebut dipisah menjadi 8 sub-node berukuran setengah dari dimensi sebelumnya pada ketiga sumbu. Jika pembagian ini dilakukan sampai ke tingkat kedalaman maksimum d, maka jumlah maksimum node yang bisa terbentuk adalah:

$$N(d) = 8^1 + 8^1 + 8^2 + \dots + 8\text{^}d$$

Ini adalah deret geometri dengan rasio 8, sehingga *worst case* jumlah total simpulnya menjadi:

$$N(d) = \frac{8^{d+1} - 1}{8-1} \approx O(8^d)$$

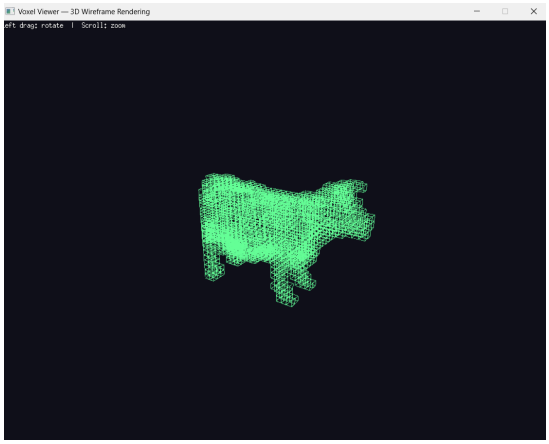
Hal ini terjadi karena setiap node memiliki tepat 8 anak potensial dan $8^d = (2^3)^d = 2^{3d}$, yang setara dengan n^3 apabila $n = 2^d$ menyatakan resolusi voxel per sumbu.

Selanjutnya, pada setiap pemanggilan makeTree, dilakukan pengecekan perpotongan terhadap seluruh T segitiga yang diwariskan dari node induk menggunakan Separating Axis Theorem (SAT) pada 13 sumbu tetap sehingga *cost* per segitiga adalah $O(1)$. Karena pengecekan ini dilakukan sebanyak T kali per node dan terdapat O^{8d} node secara keseluruhan, maka total operasi pengecekan perpotongan pada seluruh TreeNode adalah $O(T \times 8^d)$.

Dengan demikian, kompleksitas waktu total algoritma adalah $t(d) = O(T \times 8^d)$, di mana T adalah jumlah segitiga dan d adalah kedalaman maksimum octree. Pada implementasinya, kompleksitas bisa jauh lebih rendah karena terdapat penerapan *pruning* yang memangkas node-node yang tidak saling berpotongan dengan segitiga.

PENGUJIAN TEST CASE

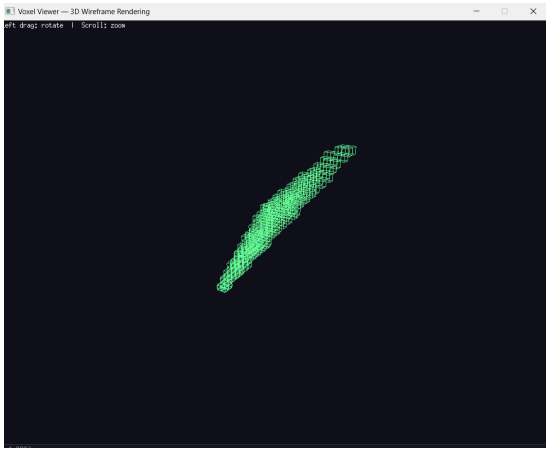
Pengujian 1 (cow.obj)

Input	Output
v 2.229345 -0.992723 -0.862826 v 2.292449 -0.871852 -0.882400 v 2.410367 -0.777999 -0.841105 v 2.407309 -0.974980 -0.805091 . . . f 3 4 2 f 1 2 4 f 5 6 3 f 4 3 6 . . .	
Voxels: 1362	

Vertices: 10896
Faces: 16344
Max depth: 6
Execution time : 72.9198 ms

Pengujian 2 (line.obj)

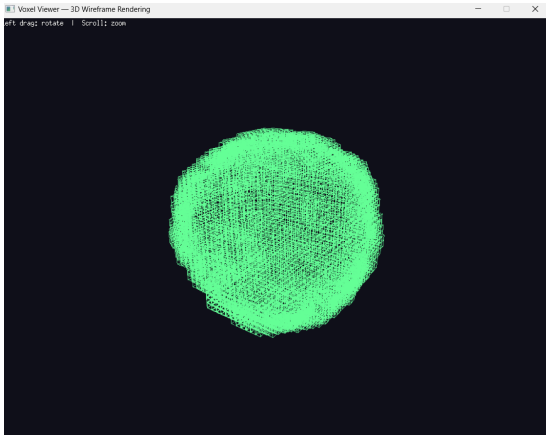
Input	Output

`v 2.229345 -0.992723 -0.862826`
`v 2.292449 -0.871852 -0.882400`
`v 2.410367 -0.777999 -0.841105`
`f 1 2 3`
	

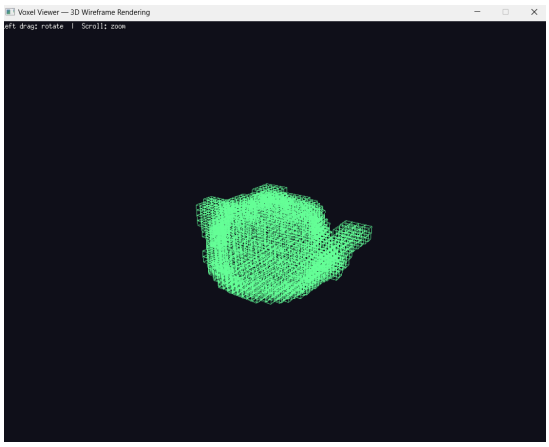
Voxels: 287	
Vertices: 2296	
Faces: 3444	
Max depth: 6	
Execution time : 4.8093 ms	

Pengujian 3 (pumpkin.obj)

Input	Output

v 3.268430 -28.849100 -135.483398 v 4.963750 -28.260300 -135.839600 v 5.762220 -27.329800 -136.463593 v 8.403750 -27.659000 -134.661789 . . . f 3971 3877 3788 f 2489 2608 2447 f 3686 3472 3473 f 2489 2533 2647 . . .	
Voxels: 4626	
Vertices: 37008	
Faces: 55512	
Max depth: 6	
Execution time : 125.6088 ms	

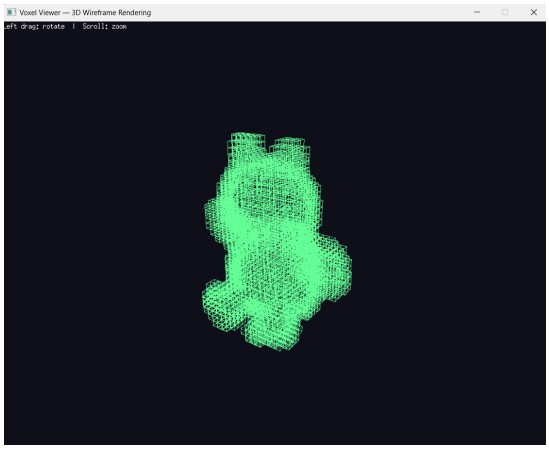
Pengujian 4 (teapot.obj)

Input	Output
v 40.6266 28.3457 -1.10804 v 40.0714 30.4443 -1.10804 v 40.7155 31.1438 -1.10804 v 42.0257 30.4443 -1.10804 . . . f 7 6 1 f 1 2 7 f 8 7 2 f 2 3 8 . . .	
Voxels: 1748	
Vertices: 13984	

Faces: 20976
Max depth: 6
Execution time : 47.1508 ms

Pengujian 5 (teddybear.obj)

Input	Output

`v -4.75738 8.49991 8.28592`
`v -4.55816 11.4788 7.74909`
`v -4.37277 13.9925 6.2828`
`v -5.70826 13.6563 6.26488`
`. . .`
`f 3 4 2`
`f 8 9 7`
`f 8 7 6`
`f 10 9 8`
`. . .`
	

Voxels: 2686	
Vertices: 21488	
Faces: 32232	
Max depth: 6	
Execution time : 202.9843 ms	

BONUS

.obj viewer

Bagian ini mengimplementasikan sebuah viewer untuk memvisualisasikan model 3D hasil voxelization secara interaktif. Pemilihan bahasa pemrograman Go dengan library Ebiten didasarkan pada efisiensinya yang tinggi dalam menangani rendering piksel sekaligus merespons input interaktif pengguna secara optimal. Selain itu, pendekatan ini diterapkan untuk mengimplementasikan logika proyeksi matematis (dari ruang 3D ke layar 2D) secara manual dari awal (from scratch), tanpa harus

mengandalkan engine 3D yang sudah jadi. Aplikasi viewer ini berfungsi sebagai antarmuka utama untuk menampilkan hasil akhir konversi objek .obj. Melalui antarmuka ini, pengguna dapat memanipulasi tampilan kamera, seperti melakukan rotasi dan penyesuaian jarak pandang (zoom), untuk memeriksa detail voxel dari berbagai sudut.

1. Struktur Data Camera dan VoxelViewer

```
// Camera defines an orbital camera system utilizing spherical
coordinates
type Camera struct {
    Yaw      float64 // Horizontal rotation
    Pitch    float64 // Vertical rotation
    Distance float64 // Radius from the target point
    Target   Vertex  // The focal point the camera orbits around
}
// VoxelViewer maintains the global state of the 3D rendering scene
type VoxelViewer struct {
    voxels     []AABB
    camera     Camera
    lastMouseX int
    lastMouseY int
    isDragging bool
}
```

Bagian ini mendefinisikan dua struktur utama untuk mengatur tampilan visual. **Camera** merepresentasikan kamera orbital yang mengelilingi objek menggunakan koordinat bola (spherical). Di dalamnya terdapat Yaw untuk rotasi horizontal, Pitch untuk rotasi vertikal, Distance untuk mengatur jarak pandang kamera, dan Target sebagai titik pusat yang menjadi fokus orbit kamera. Sementara itu, **VoxelViewer** berfungsi untuk menyimpan status global dari keseluruhan ruang 3D. Struct ini menampung daftar voxel yang akan ditampilkan, pengaturan kamera, posisi kursor mouse terakhir (lastMouseX, lastMouseY), serta penanda apakah pengguna sedang menahan dan menggeser mouse (isDragging).

2. Operasi Vektor

```go
//Vector Operations
// norm computes the unit vector
func norm(v Vertex) Vertex {
    magnitude := math.Sqrt(v.X*v.X + v.Y*v.Y + v.Z*v.Z)
    if magnitude < 1e-10 {
        return v
    }
    return Vertex{v.X / magnitude, v.Y / magnitude, v.Z / magnitude}
}
// cross computes the cross product
func cross(a, b Vertex) Vertex {
    return Vertex{
        a.Y*b.Z - a.Z*b.Y,
        a.Z*b.X - a.X*b.Z,
        a.X*b.Y - a.Y*b.X,
    }
}
// dot computes the dot product
func dot(a, b Vertex) float64 {
    return a.X*b.X + a.Y*b.Y + a.Z*b.Z
}
// subtract calculates the directional vector from b to a
func subtract(a, b Vertex) Vertex {
    return Vertex{a.X - b.X, a.Y - b.Y, a.Z - b.Z}
}
```

Keempat fungsi ini menyediakan operasi matematika vektor dasar yang sangat penting untuk menghitung posisi kamera dan melakukan proyeksi objek 3D ke layar 2D.

- **norm**: Mengubah sebuah vektor menjadi unit vector (vektor dengan panjang 1). Jika panjang vektor mendekati nol, nilainya dikembalikan apa adanya untuk mencegah error pembagian dengan nol.
- **cross**: Menghitung cross product dari dua vektor untuk menghasilkan vektor baru yang posisinya tegak lurus terhadap kedua vektor asalnya.

- **dot**: Menghitung dot product yang menghasilkan nilai skalar. Ini biasanya digunakan untuk memproyeksikan satu vektor ke arah vektor lain atau mencari sudut di antara keduanya.
- **subtract**: Mengurangi dua titik koordinat untuk mencari vektor arah, misalnya dari titik B menuju titik A.

3. Fungsi Posisi dan Basis Kamera

```
//Camera
// position the camera in x, y, z direction
func (cam *Camera) position() Vertex {
    return Vertex{
        X: cam.Target.X +
cam.Distance*math.Cos(cam.Pitch)*math.Sin(cam.Yaw),
        Y: cam.Target.Y + cam.Distance*math.Sin(cam.Pitch),
        Z: cam.Target.Z +
cam.Distance*math.Cos(cam.Pitch)*math.Cos(cam.Yaw),
    }
}
// camera movement
func (cam *Camera) basis() (right, up, forward Vertex) {
    camPos := cam.position()

    forward = norm(subtract(cam.Target, camPos))
    worldUp := Vertex{0, 1, 0}

    right = norm(cross(forward, worldUp))
    up = cross(right, forward)

    return right, up, forward
}
```

Fungsi position bertugas mengubah parameter rotasi kamera (Yaw, Pitch, Distance) menjadi koordinat 3D biasa (X, Y, Z) dengan menggunakan rumus konversi *spherical-to-cartesian*. Posisi ini dihitung secara relatif terhadap titik target yang sedang dilihat.

Selanjutnya, **fungsi basis** digunakan untuk membangun sistem koordinat lokal milik kamera itu sendiri yang terdiri dari tiga arah, yaitu forward (arah pandang ke depan), right (arah kanan), dan up (arah atas). forward didapat dengan mencari arah dari kamera ke target. right dicari dengan menyilangkan (cross product) arah forward dengan arah atas dunia nyata (0,1,0). Terakhir, up didapat dengan menyilangkan arah right dan forward. Ketiga sumbu inilah yang akan menjadi acuan saat menggambar objek ke layar.

4. Fungsi Proyeksi 3D ke 2D

```go
//3D to 2D Projection
func (viewer *VoxelViewer) project(point Vertex) (screenX float32,
screenY float32, isVisible bool) {
    right, up, forward := viewer.camera.basis()

    // Compute the direction and distance vector relative to the camera
    localPos := subtract(point, viewer.camera.position())

    camX := dot(localPos, right)
    camY := dot(localPos, up)
    depth := dot(localPos, forward)

    if depth <= 0.001 {
        return 0, 0, false
    }

    focalLength := float64(viewHeight) * focalLengthRatio

    screenX = float32(camX/depth*focalLength) + float32(viewWidth)/2
    screenY = float32(-camY/depth*focalLength) + float32(viewHeight)/2

    return screenX, screenY, true
}
```

Fungsi project adalah inti dari proses rendering yang bertugas mengubah titik 3D menjadi koordinat piksel 2D di layar monitor. Langkah pertama adalah mengubah posisi titik dari koordinat dunia nyata menjadi koordinat lokal

kamera. Hal ini dilakukan dengan mengukur posisi titik tersebut relatif terhadap sumbu kamera (camX, camY, dan kedalaman depth).

Jika nilai depth sangat kecil (≤ 0.001), artinya titik tersebut berada di belakang atau terlalu dekat dengan kamera, sehingga ditandai sebagai objek yang tidak terlihat (isVisible = false). Jika objek ada di depan kamera, posisinya di layar dihitung dengan membagi koordinat lokal dengan kedalamannya. Pembagian inilah yang menciptakan efek perspektif objek yang jauh akan terlihat lebih kecil. Sumbu Y kemudian dibalik (ditambahkan tanda minus) karena perhitungan grafis pada komputer biasanya menghitung nilai Y positif ke arah bawah layar, berbeda dengan matematika standar yang arahnya ke atas.

5. Konstruksi Geometri Box

```go
//Geometry Construction
func boxCorners(box AABB) [8]Vertex {
    return [8]Vertex{
        {box.Min.X, box.Min.Y, box.Min.Z},
        {box.Max.X, box.Min.Y, box.Min.Z},
        {box.Min.X, box.Max.Y, box.Min.Z},
        {box.Max.X, box.Max.Y, box.Min.Z},
        {box.Min.X, box.Min.Y, box.Max.Z},
        {box.Max.X, box.Min.Y, box.Max.Z},
        {box.Min.X, box.Max.Y, box.Max.Z},
        {box.Max.X, box.Max.Y, box.Max.Z},
    }
}
var boxEdges = [12][2]int{
    {0, 1}, {2, 3}, {4, 5}, {6, 7},
    {0, 2}, {1, 3}, {4, 6}, {5, 7},
    {0, 4}, {1, 5}, {2, 6}, {3, 7},
}
```

Fungsi boxCorners menguraikan batas kubus (AABB) menjadi 8 titik sudut yang presisi dengan mengombinasikan nilai minimum dan maksimum dari sumbu X, Y, dan Z. Sementara itu, **variabel boxEdges** adalah sebuah panduan yang mendefinisikan 12 rusuk pembentuk kubus. Panduan ini berisi pasangan indeks yang menunjukkan titik sudut mana yang harus dihubungkan dengan garis.

Kombinasi 8 titik sudut dan 12 garis penghubung inilah yang nantinya akan dirender sebagai sebuah gambar kerangka kubus (*wireframe*) di layar.

6. Fungsi Update (Input Handling)

```
//Ebiten Engine Interface
func (viewer *VoxelViewer) Update() error {
    mouseX, mouseY := ebiten.CursorPosition()

    if ebiten.IsMouseButtonPressed(ebiten.MouseButtonLeft) {
        if viewer.isDragging {
            deltaX := mouseX - viewer.lastMouseX
            deltaY := mouseY - viewer.lastMouseY

            viewer.camera.Yaw += float64(deltaX) * mouseSensitivity
            viewer.camera.Pitch -= float64(deltaY) * mouseSensitivity

            // Clamp pitch to prevent the camera from flipping over the
poles
            const maxPitchLimit = math.Pi/2 - 0.05
            if viewer.camera.Pitch > maxPitchLimit {
                viewer.camera.Pitch = maxPitchLimit
            } else if viewer.camera.Pitch < -maxPitchLimit {
                viewer.camera.Pitch = -maxPitchLimit
            }
        }
        viewer.isDragging = true
        viewer.lastMouseX = mouseX
        viewer.lastMouseY = mouseY
    } else {
        viewer.isDragging = false
    }

    _, scrollY := ebiten.Wheel()
    viewer.camera.Distance *= math.Pow(scrollSensitivity, scrollY)

    if viewer.camera.Distance < minCameraDistance {
        viewer.camera.Distance = minCameraDistance
```

```
    }

    return nil
}
```

Fungsi Update dipanggil secara otomatis dan terus-menerus oleh sistem Ebiten (sekitar 60 kali per detik) untuk merespons interaksi dari pengguna. Untuk fitur rotasi kamera, program akan membaca pergerakan kursor saat klik kiri mouse ditahan. Jarak geser mouse akan dikonversi menjadi sudut putar horisontal (*Yaw*) dan vertikal (*Pitch*). Nilai putaran vertikal ini juga dibatasi (*clamped*) agar kamera tidak berputar secara terbalik ketika melihat terlalu ke atas atau terlalu ke bawah (mencegah gimbal lock). Selain rotasi, fungsi ini juga menangani fitur zoom in/out dengan membaca putaran roda mouse (*scroll*). Jarak pandang ini kemudian dibatasi agar kamera tidak bisa menembus masuk terlalu dekat ke dalam objek.

7. Fungsi Draw (Rendering)

```
// Draw renders the voxel wireframes onto the screen
func (viewer *VoxelViewer) Draw(screen *ebiten.Image) {
    backgroundColor := color.RGBA{15, 15, 25, 255}
    wireframeColor := color.RGBA{80, 200, 120, 200}

    screen.Fill(backgroundColor)

    for _, box := range viewer.voxels {
        corners := boxCorners(box)

        var projectedPoints [8][2]float32
        var isVisible [8]bool

        // Project all 8 vertices of the bounding box
        for i, corner := range corners {
            screenX, screenY, visible := viewer.project(corner)
            projectedPoints[i] = [2]float32{screenX, screenY}
            isVisible[i] = visible
        }
```

```
        // Render the connected edges
        for _, edge := range boxEdges {
            startIdx, endIdx := edge[0], edge[1]

            // Only stroke the line if both vertices are in front of the
camera
            if isVisible[startIdx] && isVisible[endIdx] {
                vector.StrokeLine(
                    screen,
                    projectedPoints[startIdx][0],
projectedPoints[startIdx][1],
                    projectedPoints[endIdx][0],
projectedPoints[endIdx][1],
                    1, wireframeColor, false,
                )
            }
        }
    }

    ebitenutil.DebugPrint(screen, "Left drag: rotate  |  Scroll: zoom")
}
```

Fungsi Draw adalah eksekutor visual yang bertugas menggambar setiap frame ke layar. Prosesnya dimulai dengan mewarnai seluruh layar menjadi gelap sebagai latar belakang. Setelah itu, sistem akan mengevaluasi setiap voxel satu per satu. Delapan titik sudut dari tiap voxel akan diproyeksikan dari bentuk 3D menjadi koordinat piksel 2D menggunakan fungsi project yang sudah dibuat sebelumnya. Jika kedua ujung dari sebuah rusuk terdeteksi berada di area yang terlihat kamera (isVisible), barulah garis berwarna hijau akan digambar untuk menyambungkan kedua ujung tersebut. Proses penyaringan ini penting untuk mencegah gambar yang terdistorsi jika ada titik yang sebenarnya berada di belakang kamera. Sebagai sentuhan akhir, instruksi penggunaan kontrol dicetak di sudut layar.

8. Fungsi Layout

```
func (viewer *VoxelViewer) Layout(outsideWidth, outsideHeight int) (int,
int) {
    return viewWidth, viewHeight
}
```

Fungsi Layout digunakan oleh program Ebiten untuk menetapkan resolusi asli dari gambar yang akan dirender. Fungsi ini memastikan bahwa gambar selalu diproses pada ukuran tetap (viewWidth dan viewHeight), tidak peduli seberapa besar atau kecil ukuran jendela aplikasi ditarik oleh pengguna. Hal ini membantu menjaga konsistensi tampilan.

9. Fungsi Entry Point Viewer

```
//Entry Point
func runViewer(voxels []AABB, bounds AABB) {

    // Center the camera focus on the midpoint of the bounding volume
    center := Vertex{
        (bounds.Min.X + bounds.Max.X) * 0.5,
        (bounds.Min.Y + bounds.Max.Y) * 0.5,
        (bounds.Min.Z + bounds.Max.Z) * 0.5,
    }
    modelWidth := bounds.Max.X - bounds.Min.X

    ebiten.SetWindowSize(viewWidth, viewHeight)
    ebiten.SetWindowTitle("Voxel Viewer — 3D Wireframe Rendering")

    viewer := &VoxelViewer{
        voxels: voxels,
        camera: Camera{
            Yaw:      0.6,
            Pitch:    0.4,
            Distance: modelWidth * 2,
            Target:   center,
        },
```

```
    }

    if err := ebiten.RunGame(viewer); err != nil {
        panic(err)
    }
}
```

Fungsi runViewer adalah titik awal (entry point) di mana jendela 3D viewer ini dibangkitkan. Fungsi ini dipanggil tepat setelah proses pembuatan voxel selesai. Pertama-tama, program akan mencari titik tengah dari keseluruhan model agar kamera tahu ke mana harus fokus (Target). Lebar model juga dihitung agar sistem bisa menentukan jarak awal yang pas (*Distance*) sehingga seluruh model terlihat di layar tanpa terpotong. Kamera lalu diberikan sedikit sudut kemiringan pada Yaw dan Pitch supaya dimensi 3D-nya langsung terasa saat program dibuka. Setelah semua persiapan selesai, fungsi ebiten.RunGame akan dijalankan, yang akan memulai siklus tanpa henti dari fungsi Update (membaca input) dan Draw (menggambar layar) hingga aplikasi ditutup.

PRANALA REPOSITORY

Repository dapat diakses melalui pranala berikut:
https://github.com/hsbu/Tucil2_18223014_18223096

PERNYATAAN KEJUJURAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Matthew Sebastian Kurniawan Muhamad Hasbullah Faris

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	√	
2	Program berhasil dijalankan	√	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	√	
4	Program informasi-informasi atau report	√	
5	Program dapat menerima masukan .obj, dan mengembalikan output	√	
6	[Bonus] Program menggunakan concurrency		√
7	[Bonus] Program dapat menampilkan visualisasi .obj	√	

REFERENSI

Rinaldi Munir. (2026). "Algoritma Decrease and Conquer (Bagian 1)." Diakses pada 25 Maret 2026. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/11-Algoritma-Decrease-and-Conquer-2026-Bagian1.pdf

Rinaldi Munir. (2026). "Algoritma Decrease and Conquer (Bagian 2)." Diakses pada 25 Maret 2026. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/12-Algoritma-Decrease-and-Conquer-2026-Bagian2.pdf

Tomas Akenine-Möller. (2005). "Fast 3D Triangle-Box Overlap Testing." Diakses pada 26 Maret 2026. https://doi.org/10.1145/1198555.1198747

Geeks For Geeks. (2026). "Octree Insertion and Searching." Diakses pada 26 Maret 2026. https://www.geeksforgeeks.org/dsa/octree-insertion-and-searching/

Subodh, et al. (2000). "Interactive Visualization of Unstructured Grids Using Hierarchical 3D Voxel Octrees." Department of Computer Science and Engineering, IIT Delhi. Diakses pada 27 Maret 2026. https://www.cse.iitd.ac.in/~subodh/research/papers/gridvis.pdf